

NTSOAKI DISTRIBUTIONS

Order Management System

Software Requirements Specification (SRS)

Version	1.0
Status	Draft
Date	May 2026
Prepared by	Tau James Marake
Client	Ntsoaki Distributions

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the functional and non-functional requirements for the Ntsoaki Distributions Order Management System (OMS). It is intended to guide the design, development, and testing phases of the project, and serves as the agreement between the developer and the client on what the system must do.

1.2 Scope

The OMS is a web-based application that replaces the current spreadsheet-based order workflow. It will enable up to five concurrent users to create, manage, and track mineral distribution orders in real time. The system eliminates file-locking conflicts and version loss caused by shared OneDrive spreadsheets.

1.3 Project Context

Attribute	Detail
Organisation	Ntsoaki Distributions
Industry	Minerals Distribution (Copper and others)
Current Pain Point	Concurrent editing conflicts on shared OneDrive Excel files
Concurrent Users	Up to 5 users
Developer	Tau James Marake

1.4 Definitions

Term	Definition
OMS	Order Management System — the system being built
Order	A record of a mineral purchase or sale transaction
User	A staff member with system access
Admin	A user with full system privileges
Clerk	A user who creates and manages their own orders
Real-time	Updates visible to all users within seconds, without manual refresh
Audit Trail	A log of all changes including who made them and when

2. Overall Description

2.1 Product Overview

The OMS is a three-tier web application: a React/TypeScript frontend, a Node.js REST API backend, and a PostgreSQL database hosted on Supabase. It is accessed via a web browser and requires no software installation on user machines.

2.2 User Roles

Role	Description	Permissions
Admin	Business owner or manager	Full access: all orders, users, reports, settings
Clerk	Order entry staff	Create, view, and edit own orders; view status of others
Viewer	Read-only stakeholder	View orders and reports only

2.3 Operating Environment

- Frontend: Modern web browser (Chrome, Edge, Firefox)
- Backend: Node.js hosted on Railway or Render
- Database: PostgreSQL via Supabase
- Network: Internet connection required
- Devices: Desktop and tablet supported

2.4 Assumptions and Constraints

- Internet access is available at the Ntsoaki Distributions office.
- No more than 5 users will be active simultaneously in the initial version.
- The client will not require mobile (native app) support in Phase 1.
- Free tiers of Supabase and Railway are sufficient for early deployment.
- Existing spreadsheet data may be migrated manually; no automated migration tool is in scope.

3. Functional Requirements

3.1 Authentication & User Management

ID	Requirement
FR-01	The system shall allow users to log in using email and password.
FR-02	The system shall support three roles: Admin, Clerk, and Viewer.
FR-03	The Admin shall be able to create, deactivate, and assign roles to users.
FR-04	Sessions shall expire after 8 hours of inactivity.
FR-05	Passwords shall be stored as hashed values. No plaintext storage.

3.2 Order Management

ID	Requirement
FR-06	The system shall allow a Clerk or Admin to create a new order with fields: Order ID (auto), Client Name, Mineral Type, Quantity (kg), Unit Price (ZAR), Total, Date, and Status.
FR-07	Order status shall follow the lifecycle: Pending > Confirmed > Dispatched > Delivered > Cancelled.
FR-08	Only the order creator or an Admin may edit an order.
FR-09	Orders shall not be permanently deleted; they shall be cancelled and retained for audit purposes.
FR-10	The system shall auto-calculate the Total field from Quantity and Unit Price.
FR-11	The system shall allow attachment of a note or comment to any order.

3.3 Real-Time Collaboration

ID	Requirement
FR-12	All connected users shall see order updates within 2 seconds of a change, without page refresh.
FR-13	The system shall display a live indicator showing how many users are currently online.
FR-14	The system shall prevent two users from editing the same order simultaneously (optimistic locking or last-write-wins with conflict warning).

3.4 Dashboard & Search

ID	Requirement
FR-15	The dashboard shall display: total orders today, orders by status, total value of active orders.
FR-16	Users shall be able to filter orders by: status, date range, mineral type, and client name.

ID	Requirement
FR-17	Users shall be able to search orders by Order ID or Client Name.
FR-18	The system shall display a paginated order list, 20 records per page by default.

3.5 Reporting & Export

ID	Requirement
FR-19	The Admin shall be able to generate a summary report filtered by date range.
FR-20	Reports shall be exportable as Excel (.xlsx) and PDF.
FR-21	The system shall provide a daily order summary exportable on demand.

3.6 Audit Trail

ID	Requirement
FR-22	The system shall log every create, update, and status change with: timestamp, user, and field changed.
FR-23	The Admin shall be able to view the full activity log for any order.
FR-24	The audit log shall not be editable or deletable by any user, including Admin.

4. Non-Functional Requirements

Category	ID	Requirement
Performance	NFR-01	Page load time shall not exceed 3 seconds on a standard broadband connection.
Performance	NFR-02	Real-time updates shall reflect within 2 seconds.
Security	NFR-03	All API endpoints shall require a valid JWT token.
Security	NFR-04	All data shall be transmitted over HTTPS.
Security	NFR-05	Role-based access shall be enforced server-side, not only client-side.
Reliability	NFR-06	The system shall target 99% uptime during business hours (07:00–18:00 SAST).
Usability	NFR-07	A new user shall be able to create an order within 5 minutes without training.
Scalability	NFR-08	The database schema shall support expansion to 10+ concurrent users without redesign.
Maintainability	NFR-09	Code shall follow consistent naming conventions and include inline comments on complex logic.

5. System Architecture

5.1 Tech Stack

Layer	Technology	Purpose
Frontend	React + TypeScript + Vite	User interface
Styling	Tailwind CSS	Responsive design
State Management	React Query + Zustand	Server state and global UI state
Backend	Node.js + Express	REST API
Authentication	Supabase Auth (JWT)	Login and session management
Database	PostgreSQL (Supabase)	All persistent data
Real-time	Supabase Realtime	Live order updates
Hosting (Frontend)	Vercel	Static site deployment
Hosting (Backend)	Railway	API server deployment
Export	ExcelJS + jsPDF	Excel and PDF generation

5.2 Database Schema (Core Tables)

users

Column	Type	Notes
id	UUID	Primary key
email	VARCHAR	Unique
full_name	VARCHAR	
role	ENUM	admin clerk viewer
is_active	BOOLEAN	Soft deactivation
created_at	TIMESTAMP	Auto-set

orders

Column	Type	Notes
id	UUID	Primary key, auto-generated
order_number	VARCHAR	Human-readable e.g. ND-2026-0001
client_name	VARCHAR	
mineral_type	VARCHAR	e.g. Copper, Zinc

Column	Type	Notes
quantity_kg	DECIMAL	
unit_price_zar	DECIMAL	
total_zar	DECIMAL	Computed: quantity * unit_price
status	ENUM	pending confirmed dispatched delivered cancelled
notes	TEXT	Optional
created_by	UUID	Foreign key → users.id
created_at	TIMESTAMP	Auto-set
updated_at	TIMESTAMP	Auto-updated on change

audit_logs

Column	Type	Notes
id	UUID	Primary key
order_id	UUID	Foreign key → orders.id
changed_by	UUID	Foreign key → users.id
field_changed	VARCHAR	e.g. status, quantity_kg
old_value	TEXT	
new_value	TEXT	
changed_at	TIMESTAMP	Auto-set

6. Project Structure

6.1 Frontend (React)

```
oms-client/  src/      components/      # Reusable UI components      OrderTable.tsx
OrderForm.tsx      StatusBadge.tsx      Sidebar.tsx      pages/      # Route-
level components      Dashboard.tsx      Orders.tsx      OrderDetail.tsx
Reports.tsx      Users.tsx      Login.tsx      hooks/      # Custom React hooks
useOrders.ts      useAuth.ts      useRealtime.ts      services/      # API call
functions      ordersService.ts      reportsService.ts      store/      #
Zustand global state      authStore.ts      types/      # TypeScript interfaces
order.ts      user.ts      lib/      supabaseClient.ts      queryClient.ts      App.tsx
main.tsx
```

6.2 Backend (Node.js + Express)

```
oms-api/  src/      routes/      # Express route definitions
orders.routes.ts      users.routes.ts      reports.routes.ts      auth.routes.ts
controllers/      # Request handlers      orders.controller.ts
users.controller.ts      reports.controller.ts      middleware/      auth.middleware.ts
# JWT verification      role.middleware.ts      # Role-based access      services/
# Business logic      orders.service.ts      audit.service.ts      export.service.ts
db/      supabase.ts      # Supabase client init      utils/      orderNumber.ts      #
ND-YYYY-XXXX generator      app.ts      server.ts
```

7. Development Phases

Phase	Scope	Est. Duration
Phase 1 — Core	Auth, order CRUD, status lifecycle, basic dashboard	3–4 weeks
Phase 2 — Collaboration	Real-time updates, online indicator, conflict handling	1–2 weeks
Phase 3 — Reporting	Excel/PDF export, summary reports, date filters	1–2 weeks
Phase 4 — Audit & Polish	Full audit trail, UI refinement, UAT with client	1 week

8. Out of Scope (Version 1.0)

- Native mobile application (iOS / Android)
- Automated migration of existing Excel data
- Integration with accounting software (e.g. Sage, QuickBooks)
- Inventory or stock management
- Customer-facing portal
- Automated email notifications (deferred to v1.1)

9. Acceptance Criteria

The system will be accepted when all of the following are verified:

#	Criterion
AC-01	Five users can log in simultaneously and create orders without conflict.
AC-02	Order updates appear on all connected screens within 2 seconds.
AC-03	An order created by Clerk A cannot be edited by Clerk B.
AC-04	An exported Excel file contains accurate, untruncated order data.
AC-05	The audit log correctly records all status changes with user and timestamp.
AC-06	Cancelling an order does not delete it from the database.
AC-07	A non-Admin user cannot access the Users management page.

End of Document — Ntsoaki Distributions OMS SRS v1.0